

Ministry of Education and Science of Ukraine
National Technical University of Ukraine
"Kyiv Polytechnic Institute named after Igor Sikorsky"
Educational and Scientific Institute of Atomic and Thermal Energy
Department of Digital Technologies in Energy

Graphics work

Topic: «Spatial audio»

from the discipline: «Methods of synthesis of virtual reality»

Variant 10

Performed by:

student of the TP-52mp group

Striltsov D.S.

Kyiv – 2026

1.Task

1.1 Project Objective

The goal of this project is to develop an interactive web application that combines stereo 3D rendering with spatial audio control. The application must allow the user to manipulate a virtual audio source position while keeping the visual surface static. The spatial sound should be processed using the Web Audio API, including a band-pass filter.

1.2 Requirements

The main tasks for the project are:

- Implement stereo rendering of a 3D scene with adjustable camera parameters.
- Present a fixed surface and a separate audio source sphere in the rendered scene.
- Use user-controlled yaw to reposition the sound source around the surface.
- Build a Web Audio API graph with `MediaElementSourceNode`, `PannerNode`, and `BiquadFilterNode`.
- Organize the user interface so spatial audio controls are visually separated from stereo rendering controls.
- Ensure the app runs in a browser with WebGL and Web Audio support.

1.3 Scope of Work

The implementation includes:

- Core rendering logic and scene updates in `main.js`.
- User interface layout and controls in `index.html`.
- Stereo camera calculations via `StereoCamera.js`.
- Matrix math functions in `Utils/m4.js`.
- Basic shader logic in `shader.gpu`.

2. Theoretical information

2.1 Stereo Rendering Theory

Stereo rendering is a process by which two images are generated for the left and right eyes, creating the perception of depth. In a web application, stereo rendering can be implemented by rendering the same scene from two slightly shifted camera positions. Each camera uses a separate projection matrix, and the final output is shown side by side or composited for a stereoscopic display.

Key elements of stereo rendering:

- Eye separation: the distance between the left and right camera positions.
- Convergence plane: the distance at which both views intersect, controlling perceived depth.
- Field of view: the vertical angle of the camera frustum.
- Near and far clipping planes: boundaries that define the visible depth range.

In this project, stereo rendering is constructed by updating the camera frustums for left and right eyes and by blending the two renderings into a single presentation.

2.2 Web Audio API Fundamentals

The Web Audio API provides a powerful audio processing graph for interactive audio on the web. A minimal spatial audio setup includes:

- `AudioContext`: the audio processing environment.
- `HTMLMediaElement`: an audio or video element that can be used as a source.
- `MediaElementSourceNode`: connects the media element to the audio graph.
- `PannerNode`: positions audio in 3D space.
- `BiquadFilterNode`: applies frequency filtering such as low-pass, high-pass, or band-pass.
- `GainNode`: controls volume levels in the graph.

Spatial audio uses distance and panning models to simulate realistic source placement. The `PannerNode` can use `HRTF` or `equalpower` panning models. In this project, the audio source position is updated dynamically in 3D coordinates, while the listener is fixed at the scene center.

2.3 Band-pass Filtering

A band-pass filter allows frequencies within a target range to pass while attenuating frequencies outside that range. In the Web Audio API, a `BiquadFilterNode` is configured with a filter type of `'bandpass'`.

Important parameters:

- 'frequency': the center frequency of the pass band.
- 'Q': the quality factor, which controls filter sharpness.

A band-pass filter enhances the spatial audio experience by focusing the source sound within a controlled spectral range.

2.4 Interaction and User Controls

A simple and robust user interface is critical for usability. The project separates rendering controls from audio controls, allowing users to manage each subsystem independently.

Implemented controls include:

- Stereo configuration sliders for field of view and clipping planes.
- Toggle switches for wireframe and filled polygon rendering.
- A file input for loading the audio track.
- Filter enable/disable and parameter adjustments.
- Manual yaw slider for audio source rotation.
- Play/pause and reset buttons for audio control.

These controls reflect a direct manipulation style that is easy to understand and operate.

3. Implementation Details

3.1 Code Organization

The application is organized into the following components:

Module	Description
Model.js	Generation of parametric geometry, normals, tangents, and buffers
StereoCamera.js	Stereo camera view setup and frustum control.
main.js	Application logic, rendering loop, event handling, and audio graph management.
shader.gpu	Shader source code for rendering.
index.html	UI elements and page structure.

3.2 Rendering Pipeline

The rendering pipeline follows these steps:

1. Initialize WebGL and compile shaders.
2. Set up geometry buffers for the surface and the sound source sphere.
3. Create a stereo camera instance and configure left/right frustums.
4. Render the scene into the canvas on each animation frame.
5. Update the view matrices based on user inputs and model transform settings.

The visual surface remains static, while the sound source sphere rotates using the yaw angle controlled by the user.

3.3 Audio Graph Implementation

The audio graph is built in `main.js` as follows:

1. Create an `AudioContext`.
2. Load an audio file via `HTMLAudioElement`.

3. Create a `'MediaElementSourceNode'` from the audio element.
4. Create a `'BiquadFilterNode'` and set it to `'bandpass'`.
5. Configure a `'PannerNode'` with spatial position updates.
6. Connect the nodes: source -> filter -> panner -> destination.

The filter settings and panner position are updated in response to UI events. The yaw slider changes the audio source position around the scene center.

3.4 User Interface Layout

The main interface is split into three sections:

- Stereo controls block
- Spatial audio controls block
- Information and instructions block

This layout improves clarity and keeps audio controls distinct from visual configuration.

The audio block includes:

- Audio file input
- Filter enable checkbox
- Slider controls for filter frequency and Q
- Manual yaw slider
- Play/pause button
- Reset source button

The UI block separation ensures that users can easily find the spatial audio settings without searching through stereo options.

4. User's instruction

4.1 Starting the Application

To begin working with the application, open the project folder in a browser or via a local server. Then open the index.html file in a browser that supports WebGL and Web Audio technologies. If prompted, allow audio playback.

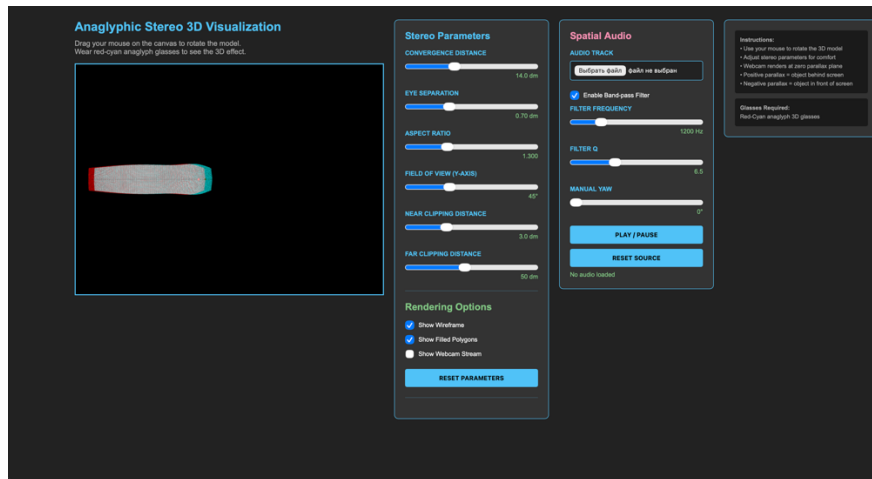


Figure 1 – Initial layout of the application with all main control sections visible.

4.2 Loading an Audio Track

Click the Audio Track file input, then select an audio file in one of the supported formats (MP3, WAV, or OGG). After successful loading, the application will display the message “Audio loaded” and enable the filter controls.

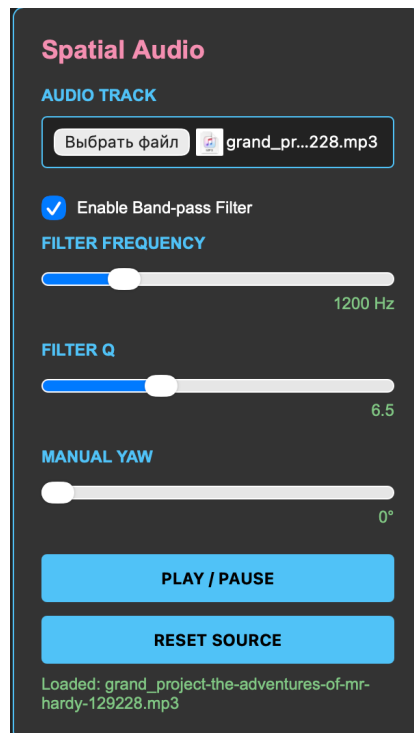


Figure 4.2 – Audio track loaded with filter controls visible.

4.3 Controlling Spatial Audio

Once the audio track is loaded, you can control the spatial audio as follows:

- Use the Manual Yaw slider to rotate the audio source around the fixed surface.
- Toggle the Enable Band-pass Filter checkbox to activate or deactivate the filter. Adjust the Filter Frequency and Filter Q sliders to shape the sound.
- Press the Play / Pause button to start or stop playback.

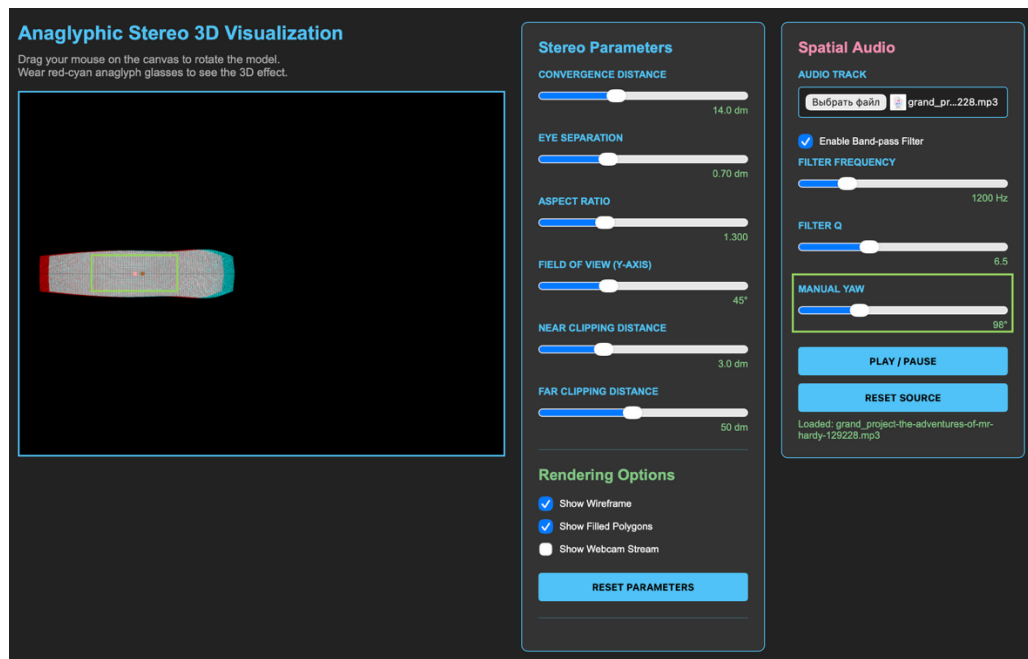


Figure 4.3 – Manual Yaw slider and source movement in the 3D scene.

4.4 Reset and Visual Feedback

Click the Reset Source button to return the audio source to its default position. Monitor the status label at the bottom of the audio section for real-time messages. Additionally, you can adjust the stereo rendering options to optimize the 3D visual representation.

5. Source code

The following example illustrates how the audio graph and source control are implemented in `main.js`.

```
function createAudioContext() {
  const audioContext = new AudioContext();

  const filterNode = audioContext.createBiquadFilter();
  filterNode.type = 'bandpass';
  filterNode.frequency.value = 1200;
  filterNode.Q.value = 6.5;

  const pannerNode = audioContext.createPanner();
  pannerNode.panningModel = 'HRTF';
  pannerNode.distanceModel = 'inverse';
  pannerNode.setPosition(0, 0, -10);

  sourceNode.connect(filterNode);
  filterNode.connect(pannerNode);
  pannerNode.connect(audioContext.destination);

  return { audioContext, filterNode, pannerNode };
}

function updateSourcePosition(yawDegrees) {
  const yaw = (yawDegrees * Math.PI) / 180;
  const radius = 6.0;
  const x = Math.sin(yaw) * radius;
  const z = Math.cos(yaw) * radius;
  pannerNode.setPosition(x, 0, z);
}

manualYaw.addEventListener('input', (event) => {
  const yawValue = parseFloat(event.target.value);
  updateSourcePosition(yawValue);
  manualYawValue.textContent = yawValue.toFixed(0);
});
```

Additional rendering code sets up the stereo camera and draws the sphere representing the sound source. These code sections are located in `main.js` and `StereoCamera.js`.